3-Inter-Process Communication

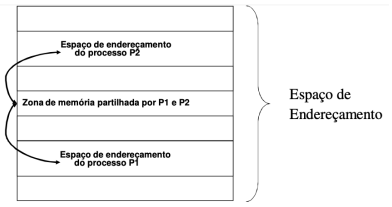
1

Channel Implementation

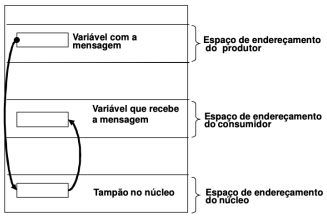
- To implement the communication channel it is essential to define how data is transferred between the address spaces of the processes
- The communication channel can be implemented with two types of mechanisms:
  - **Shared memory**: processes access a memory area that is part of the address space dos communicating processes
  - Transferred through the **kernel do operating system**; data is always copied to the kernel before being transferred

5

Channel Implementation



Shared Memory



With Copy to Kernel

- Shared memory can be considered as an extension to virtual memory management, allowing the address space protection mechanisms to be bypassed
- Communication in the kernel is similar to the management of other system objects, particularly files

6

Channel Implementation

- Shared memory:
  - most efficient mechanism
  - synchronization must be explicitly programmed
  - complex programming
- System Communication Object:
  - transfer speed limited by the two copies of the data and the use of system calls to Send and Receive
  - implicit synchronization
  - easier to use

7

Inter-process communication em UNIX

- In UNIX there was an attempt to standardize the inter-process communication interface with the file system interface.
- To understand the communication mechanisms it is essential to know the file system interface well.

File System

- Hierarchical file system
- File types:
  - **Regular** – byte sequence (bytes) without organization in records (*records*)
  - **Special files** – I/O peripherals, pipes, fifos, sockets
  - **Directory files**
- When a process starts executing the system opens three special files
  - ***stdin*** – Input to the process (fd – 0)
  - ***stdout***– Output from the process (fd – 1)
  - ***stderr*** – peripheral to signal errors (fd – 2)
- A file descriptor is an integer used to identify an open file (values range from zero to a system-dependent maximum)

File System

```
int main (int argc, char *argv[])
{
  int source, destination, n;
  char buffer[1024];

  source = open (argv[1], O_RDONLY);
  if (source == -1) {
   perror ("Opening Source File");
   exit(1);
  }

  destination = open(argv[2], O_CREAT | O_TRUNC | O_WRONLY, 0666);
  if (destination == -1) {
        perror ("Opening Destination File");
   exit(1);
  }

  while ((n = read (source, buffer, sizeof(buffer))) > 0)
   write (destination, buffer, n);

  exit(0);
}
```


10

IPC in UNIX

- *pipes*
- *named pipes* (FIFOS)
- *UNIX domain sockets*
- *Internet sockets*
- Shared memory regions

12

Pipes

- Original UNIX mechanism for inter-process communication.
- They have an interface identical to that of files
- Channel (*byte stream*) connecting two processes
- Allows a unidirectional information flow, one process writes to a *pipe* and the corresponding process reads from the other end – one-to-one model
- Has no logical name associated
- Messages are sequences of *bytes* of any size



Pipes

```
int pipe (int *fds);

   fds[0] – descritor aberto para leitura
   fds[1] – descritor aberto para escrita
```

- Descriptors of a *pipe* are analogous to file descriptors
- The operations of *read* and *write* on files are valid for *pipes*
- Descriptors are local to a process and can be transmitted to child processes through the inheritance mechanism
- The process is blocked when writing to a full *pipe*
- The process is blocked when reading from an empty *pipe*



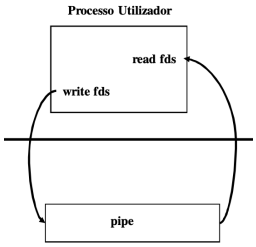
Pipes

```
char msg[] = "utilizacao de pipes";

main() {
    char tampao[1024];
    int fds[2];

    pipe(fds);

    for (;;) {
       write (fds[1], msg, sizeof (msg));
       read (fds[0], tampao, sizeof (msg)));
    }
}
```



Pipes

```
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>

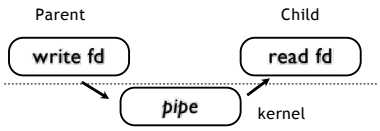
#define TAMSG 100

char msg[] = "test message";
char tmp[TAMSG];

main() {
   int fds[2], pid_child;

   if (pipe (fds) < 0) exit(-1);

   if (fork () == 0) { /* read from pipe */
    read (fds[0], tmp, sizeof (msg));
    printf ("%s\n", tmp);
    exit (0);
   }
   else {
    write (fds[1], msg, sizeof (msg));
    wait(NULL);
   }
}
```



Input/Output Redirection

```
DUP(2)                    BSD System Calls Manual                    DUP(2)

NAME
     dup, dup2 -- duplicate an existing file descriptor

SYNOPSIS
     #include <unistd.h>

     int
     dup(int fildes);

     int
     dup2(int fildes, int fildes2);

DESCRIPTION
     Dup() duplicates an existing object descriptor and returns its value to
     the calling process (fildes2 = dup(fildes)).  The argument fildes is a
     small non-negative integer index in the per-process descriptor table.
     The value must be less than the size of the table, which is returned by
     getdtablesize(2).  The new descriptor returned by the call is the lowest
     numbered descriptor currently not in use by the process.
```


Input/Output Redirection

```c
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>

#define TAMSG 100

char msg[] = "test message";
char tmp[TAMSG];

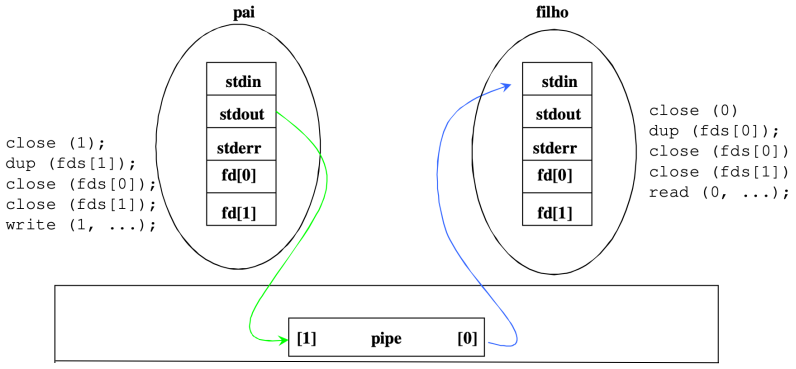
main() {
    int fds[2], pid_child;

    if (pipe (fds) < 0) exit(-1);

    if (fork () == 0) {
     /* release stdin (position zero) */
     close(0);
     /* redirect stdin to the read pipe */
     dup (fds[0]);
     /* close descriptors not used by the child */
     close (fds[0]);
     close (fds[1]);

     /* read from pipe */
     read (0, tmp, sizeof (msg));
     printf ("%s\n", tmp);
     exit (0);
    }
    else {
         /* release stdout (position one) */
         close(1);
         /* redirect stdout to the write pipe */
         dup (fds[1]);
         /* close descriptors not used by the parent */
         close (fds[0]);
         close (fds[1]);
         write (1, msg, sizeof (msg));
         wait(NULL);
        }
}
```

Input/Output Redirection



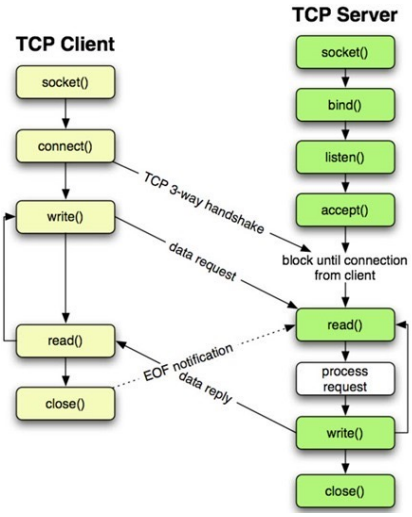
Named pipes (FIFOS)

- For two processes (that are not parent and child) to communicate, the *pipe* must be identified by a name
- A logical name is assigned to the *pipe*. The namespace used is that of the file system
- A *named pipe* behaves externally like a file, with an entry in the corresponding directory
- A *named pipe* can be opened by processes that have no hierarchical relationship
- See example (fifo-srv.c, fifo-clt.c)

Internet Sockets

- Um *Internet socket* ou *network socket* is a communication mechanism between processes that may be running on different machines connected by a data network
- Communication is bidirectional and supports different types of communication semantics (SOCK_STREAM, SOCK_DGRAM e SOCK_RAW)
- The API is shown in the following figure.
- See example is-server.c and is-client.c

Internet Sockets



Unix Domain Sockets

- Um UNIX domain socket ou IPC socket is a communication mechanism between processes running on the same operating system
- Communication is bidirectional and supports different types of communication semantics (SOCK_STREAM, SOCK_DGRAM e SOCK_RAW)
- The API is very similar to that of *Internet Sockets*, however, in this case a protocol stack TCP/IP is not used.
- see example uds-server.c and uds-client.c

Shared Memory

- Allows multiple processes to access a common memory area
- The segment size cannot be changed after creation
- Each process can "ver" the segment at different addresses in its address space
- Creating a region:

```
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/shm.h>
int shmget (key_t key, int size, int shmflg)
```

- `size` especifica the size of the region in bytes

Shared Memory

- Attaching to a region:
  - `char* shmat (int shmid, char *shmaddr, int shmflg)`
- Returns the base address of the region
- The address can:
  - ser especificado por `shmaddr`
  - se `shmaddr` for zero,o endereço é calculado pelo sistema
- Se `shmflg = SHM_RDONLY` o acesso fica restrito a leitura
- Detaching from region:
  - `int shmdt (char *shmaddr);`
- See Example (shmem-clt.c shmemsrv.c)